

Лабораторная работа № 9

Тема: Настройка Linux-маршрутизатора, nftables, NAT, port forwarding и WireGuard

Цель: Приобрести практические навыки настройки Linux-сервера как маршрутизатора, включения IP forwarding, применения правил nftables, настройки NAT masquerade, проброса порта и базовой проверки WireGuard VPN.

Ориентировочное время: 4-6 академических часов

Среда: VirtualBox

ОС: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

1. Необходимые ресурсы

- `gw01` - Linux-router с двумя сетевыми адаптерами: внешняя сеть и внутренняя LAN;
- `web01` - внутренний web-сервер в LAN;
- `external01` - внешний клиент для проверки port forwarding;
- `vpn-client` - клиент WireGuard или вторая Linux-ВМ для проверки туннеля;
- пакеты `nftables`, `wireguard`, `curl`, `iproute2`, `tcpdump` по возможности;
- доступ к репозиториям через NAT или заранее подготовленные пакеты;

- методичка `methodical_guide.md`.

2. Профессиональная ситуация

Небольшой офис использует Linux-сервер как шлюз между внутренней сетью и внешним сегментом. Нужно обеспечить выход LAN-клиентов наружу через NAT, опубликовать внутренний web-сервер на внешнем порту и подготовить защищённый VPN-доступ для удалённого клиента. Firewall должен работать по принципу запрета по умолчанию с явными разрешениями.

3. Исходные данные и ограничения

Параметр	Значение
Маршрутизатор	<code>gw01</code>
Внешний интерфейс	<code>enp0s3</code>
Внешний адрес <code>gw01</code>	<code>192.168.100.2/24</code>
Внутренний интерфейс	<code>enp0s8</code>
Внутренний адрес <code>gw01</code>	<code>192.168.10.1/24</code>
LAN	<code>192.168.10.0/24</code>

Web-сервер	web01, 192.168.10.20/24, gateway 192.168.10.1
Внешний клиент	external01, 192.168.100.100/24
Port forwarding	192.168.100.2:8080 -> 192.168.10.20:80
WireGuard-интерфейс	wg0
VPN-сеть	10.8.0.0/24
VPN-адрес gw01	10.8.0.1/24
VPN-адрес клиента	10.8.0.2/32
WireGuard-порт	51820/udp

Имена интерфейсов могут отличаться. Перед настройкой определите реальные имена через `ip link` и используйте их последовательно во всей работе.

4. Требования безопасности

Важно: перед включением `policy drop` в `firewall` сделайте снимок ВМ gw01 и убедитесь, что есть консольный доступ. Ошибка в `nftables` может заблокировать удалённое управление.

Соблюдайте требования:

- не применяйте deny by default до добавления правил для loopback, established/related и нужного управления;
- проверяйте `nft list ruleset` после каждого блока правил;
- включайте port forwarding только на конкретный адрес и порт;
- не публикуйте административные сервисы через DNAT;
- не прикладывайте закрытые ключи WireGuard к отчёту;
- защищайте `/etc/wireguard/wg0.conf` правами `600`.

5. Порядок выполнения работы

Этап 1. Подготовьте адресацию и маршруты

Настройте адреса интерфейсов `gw01`, `web01` и `external01`. На `web01` укажите шлюз по умолчанию `192.168.10.1`.

Контрольные точки:

```
ip addr
ip route
ping -c 4 192.168.10.20
```

Критерий перехода: `gw01` видит внутренний web-сервер и внешний сегмент, `web01` использует `gw01` как gateway.

Этап 2. Включите IPv4 forwarding

Включите пересылку IPv4-пакетов временно и постоянно через `sysctl`.

Контрольные точки:

```
sysctl net.ipv4.ip_forward
sudo sysctl --system
```

Критерий перехода: `net.ipv4.ip_forward = 1`.

Этап 3. Настройте базовый firewall nftables

Создайте таблицу `inet filter` с цепочками `input`, `forward` и `output`.

Настройте policy `drop` для входящего и транзитного трафика, но явно разрешите:

- loopback;
- established/related;
- SSH или консольное управление из LAN, если используется;
- транзит LAN -> WAN;
- ответы WAN -> LAN через connection tracking;
- UDP `51820` на внешнем интерфейсе для WireGuard.

Контрольная точка:

```
sudo nft list ruleset
```

Критерий перехода: правила присутствуют, управление `gw01` не потеряно.

Этап 4. Настройте NAT masquerade для LAN

Создайте таблицу `ip nat`, цепочку `postrouting` и правило masquerade для трафика, выходящего через внешний интерфейс.

Контрольные точки:

```
sudo nft list ruleset  
ping -c 4 192.168.100.100
```

Критерий перехода: LAN-трафик проходит через `gw01`, правило `masquerade` присутствует в `postrouting`.

Этап 5. Проверьте выход внутреннего узла наружу

С `web01` проверьте маршрут по умолчанию и доступность внешнего клиента или учебного внешнего адреса. Если есть внешний HTTP-сервис, проверьте доступ через `curl`.

Контрольные точки:

```
ip route
ping -c 4 192.168.100.100
```

Критерий перехода: `web01` отправляет трафик наружу через `192.168.10.1`.

Этап 6. Настройте port forwarding на web-сервер

Опубликуйте внутренний HTTP-сервер `web01:80` на внешнем адресе `gw01` через порт `8080`. Настройка должна включать DNAT в `prerouting` и разрешающее правило в цепочке `forward`.

Контрольная точка с `external01`:

```
curl -I http://192.168.100.2:8080
```

Критерий перехода: внешний клиент получает HTTP-ответ от внутреннего `web01`.

Этап 7. Настройте WireGuard VPN

На `gw01` и `vpn-client` установите WireGuard, создайте пары ключей и настройте интерфейс `wg0`. На сервере используйте адрес `10.8.0.1/24`, на

клиенте - 10.8.0.2/32. В AllowedIPs клиента включите доступ к 192.168.10.0/24.

Обязательные параметры:

- ListenPort = 51820 на gw01;
- public key клиента в секции [Peer] сервера;
- public key сервера в секции [Peer] клиента;
- firewall-правила между wg0 и enp0s8.

Контрольные точки:

```
sudo wg  
ping -c 4 10.8.0.1  
ping -c 4 192.168.10.20
```

Критерий перехода: есть handshake WireGuard, VPN-клиент достигает внутреннего web-сервера.

Этап 8. Сохраните правила и проверьте автозапуск

Настройте загрузку nftables и WireGuard после перезагрузки. Проверьте, что конфигурация не зависит только от ручных команд текущего сеанса.

Контрольные точки:

```
sudo systemctl enable nftables  
sudo systemctl enable wg-quick@wg0  
sudo nft list ruleset  
systemctl status wg-quick@wg0
```

Критерий перехода: правила и VPN-интерфейс могут быть восстановлены после перезагрузки.

Этап 9. Выполните диагностику типовой ошибки

Смоделируйте одну безопасную проблему: отключённый `ip_forward`, отсутствие forward-правила, неверный gateway на `web01`, неправильный `AllowedIPs` или закрытый UDP `51820`. Найдите причину и восстановите работу.

Диагностические команды:

```
ip route
sysctl net.ipv4.ip_forward
sudo nft list ruleset
sudo wg
journalctl -u wg-quick@wg0 -n 50 --no-pager
```

Контрольная точка: в отчёте есть симптом, диагностическая команда, причина и исправление.

6. Итоговая проверка

Убедитесь, что:

- `gw01` имеет внешний и внутренний интерфейсы;
- `net.ipv4.ip_forward` равен `1`;
- `nftables` содержит filter-цепочки и NAT-цепочки;
- LAN-трафик проходит через `masquerade`;
- port forwarding `192.168.100.2:8080 -> 192.168.10.20:80` работает;
- WireGuard слушает UDP `51820`;
- VPN-клиент имеет handshake с `gw01`;
- VPN-клиент достигает `192.168.10.20`;
- закрытые ключи не раскрыты в отчёте;

- диагностическое действие выполнено.

7. Паспорт маршрутизатора и VPN

Параметр	Значение на стенде
Имя маршрутизатора	gw01
Внешний интерфейс и адрес	
Внутренний интерфейс и адрес	
LAN-сеть	192.168.10.0/24
Значение <code>net.ipv4.ip_forward</code>	
Правило NAT	
Правило port forwarding	
Адрес web-сервера	192.168.10.20
Внешняя проверка HTTP	
WireGuard-интерфейс	wg0
VPN-адрес сервера	10.8.0.1/24

VPN-адрес клиента	10.8.0.2/32
Команда проверки WireGuard	

8. Что приложить к отчёту

Приложите 6-8 доказательств:

1. Схему стенда с внешней сетью, `gw01`, LAN, `web01` и VPN-клиентом.
2. Вывод `ip addr` и `ip route` на `gw01`.
3. Вывод `sysctl net.ipv4.ip_forward`.
4. Вывод `nft list ruleset` с filter, NAT, masquerade и DNAT.
5. Проверку выхода LAN-узла наружу.
6. Вывод `curl -I http://192.168.100.2:8080` с внешнего клиента.
7. Вывод `wg` без закрытых ключей, подтверждающий handshake.
8. Описание диагностированной ошибки и способа исправления.

9. Критерии успешного выполнения

Работа выполнена успешно, если:

- Linux-сервер выполняет роль маршрутизатора между двумя сетями;
- IPv4 forwarding включён постоянно;
- nftables работает по принципу запрета по умолчанию;
- NAT masquerade обеспечивает выход LAN наружу;

- port forwarding публикует только нужный порт web-сервера;
- WireGuard-туннель поднимается и даёт доступ к LAN;
- правила и VPN сохраняются после перезагрузки;
- отчёт не содержит закрытых ключей;
- студент может объяснить ip_forward, forward-chain, masquerade, DNAT, AllowedIPs и handshake.

10. Контрольные вопросы

9. Зачем Linux-маршрутизатору параметр `net.ipv4.ip_forward`?
10. Чем цепочка `input` отличается от цепочки `forward` в firewall?
11. Почему подход deny by default безопаснее для маршрутизатора?
12. Чем masquerade отличается от обычного SNAT?
13. На каком hook выполняется DNAT для port forwarding?
14. Почему DNAT без разрешающего правила `forward` не обеспечивает доступ к web-серверу?
15. Что означает AllowedIPs в WireGuard?
16. Что проверить, если WireGuard handshake есть, но VPN-клиент не видит LAN?